Multi-Elevator Dispatching with Tabular Q-Learning

Results of the experiments. Every number here is produced by *run_experiments.py* and this report by *make_report.py* from the saved results.

1. Setup

Simulator. Discrete time: a car moves one floor per tick and a stop takes 2 ticks. Each car follows LOOK on its own stops. Passengers arrive as a Poisson process; the arrival rate is scaled with building height and number of cars so the load per car is comparable (intensity 2.4). Car capacity is 8. Traffic profiles: up-peak (85% of trips from the lobby), down-peak (85% to the lobby) and interfloor (uniform).

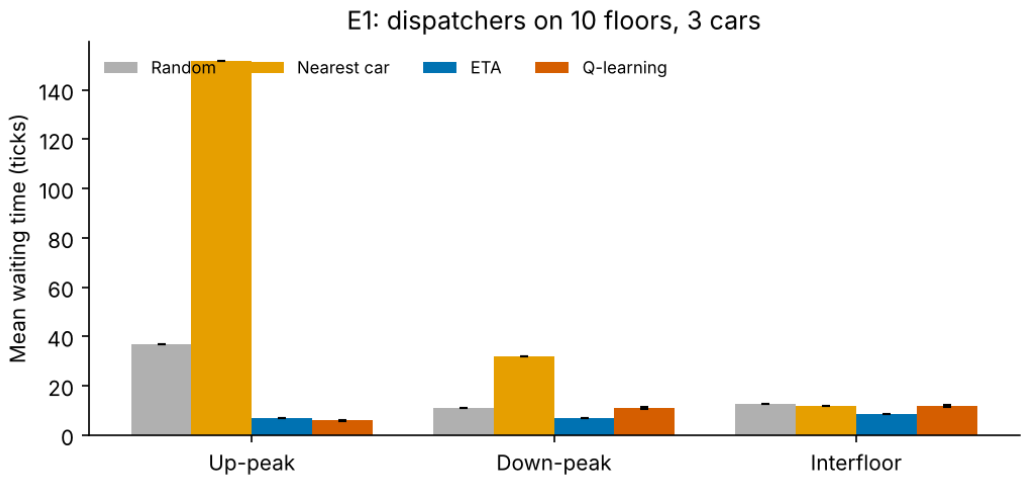
Decision problem. Whenever a new hall call appears, a dispatcher chooses which car takes it. Baselines: **Random**; **Nearest car** (closest car by floor, ties broken at random); **ETA** (car with the smallest estimated time of arrival following LOOK, the standard industry rule).

MDP and Q-learning. State: whether the call is at the lobby, its direction, how many hall calls are open (4 levels), whether anyone has waited over 40 ticks, and each car's type = (ETA bucket, busyness bucket), listed as a sorted multiset because the cars are interchangeable. Action: which car type to send. Reward: minus the waiting cost paid until the next decision. Per tick each waiting passenger costs (1 - lambda) + lambda (2w + 1)/20, where w is their current wait; summed over a wait this gives the total wait when lambda = 0 and the total squared wait (scaled) when lambda = 1. Update: Q(s,a) <- Q(s,a) + alpha [R + gamma^{tau} max Q(s',a') - Q(s,a)], where tau is the number of ticks between decisions. Epsilon-greedy exploration decays from 0.3 to 0.02; alpha = max(0.02, n$^{-0.6}$) for the n-th visit of (s, a).

Protocol. 3000 training episodes of 800 ticks per run, 3 independent training seeds per setting (reported as mean +/- std across seeds). Every method is tested on the same 20 unseen passenger streams; passengers arriving in the first 100 ticks are ignored. People still waiting at the end count with the wait so far. In test states never seen during training, the Q-learning dispatcher falls back to ETA; the share of decisions where this did not happen is reported as coverage. Waiting time is measured in ticks.

2. Results

E1. Traffic patterns (10 floors, 3 cars)

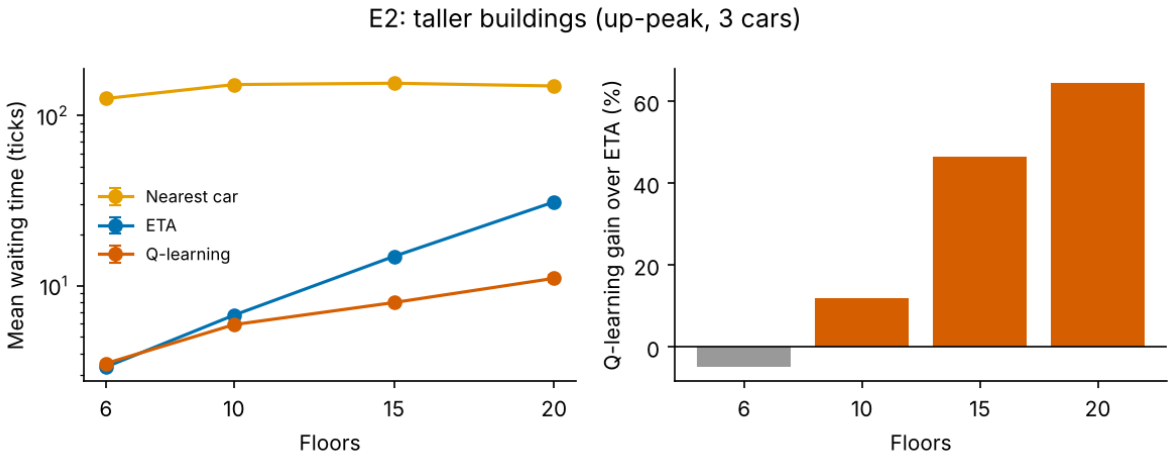


Profile	Random	Nearest car	ETA	Q-learning	Gain vs ETA
Up-peak	36.8	151.9	6.7	5.9 +/- 0.2	+12%
Down-peak	11.0	32.1	6.7	10.8 +/- 0.4	-61%
Interfloor	12.7	11.7	8.4	11.8 +/- 0.2	-40%

Under up-peak traffic Q-learning beats ETA: mean wait 5.89 +/- 0.16 vs 6.70 ticks (+12%), and 95th percentile 16.3 vs 19.6. The gain holds in all three seeds, but its size at 10 floors was sensitive to small corrections to the

training code during development (it ranged from about +4% to +12%), so we read it as a modest advantage rather than a precise figure. Under down-peak and interfloor traffic it is clearly worse than ETA (10.8 vs 6.7 and 11.8 vs 8.4 ticks), close to the Random dispatcher (11.0 and 12.7). Our explanation is credit assignment. The reward is the waiting cost of everyone in the building. In up-peak most calls come from the lobby, the same states recur, and each decision decides where a whole group goes. When calls are spread over all floors, each decision affects one or two people, and that small effect is buried in the random fluctuation of the building-wide cost. The diagnostic in analyze_policy.py supports this: in well-visited states the gap between the best and worst available action's value has a median of 0.6% of the value under interfloor traffic, against 1.9% under up-peak, so in interfloor traffic the greedy choice is close to a coin toss. Nearest car does badly in up-peak because it ignores the stops a car has already committed to.

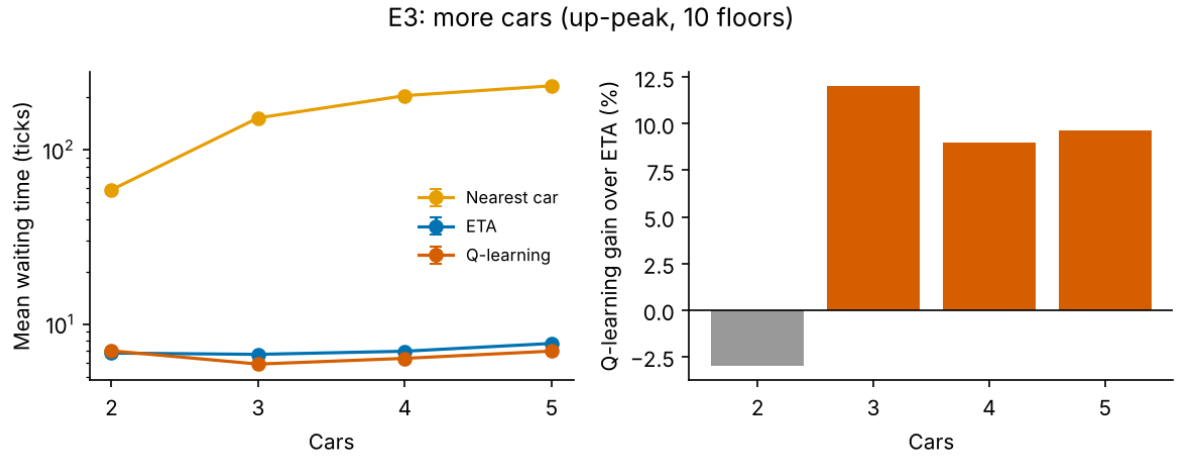
E2. Building height (up-peak, 3 cars)



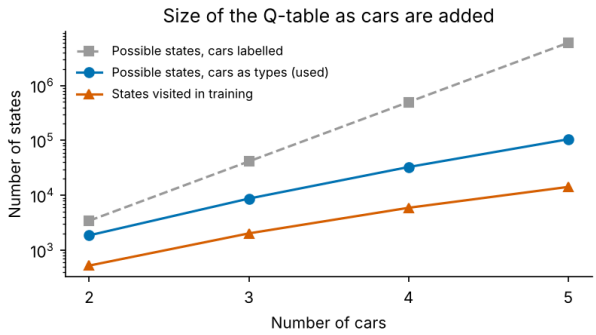
Floors	Nearest car	ETA	Q-learning	Gain vs ETA	Q p95	ETA p95
6	125.7	3.3	3.5 +/- 0.1	-5%	9.2 +/- 0.4	9.4
10	151.9	6.7	5.9 +/- 0.2	+12%	16.3 +/- 0.7	19.6
15	154.6	14.8	7.9 +/- 0.2	+46%	20.2 +/- 1.2	41.9
20	149.0	31.0	11.0 +/- 0.5	+64%	28.0 +/- 2.3	74.4

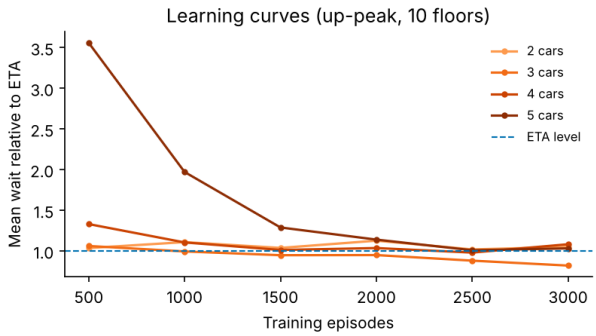
The advantage of learning grows with building height: from -5% at 6 floors to +64% at 20 floors. ETA's mean wait rises from 3.3 to 31.0 ticks while Q-learning's rises only from 3.5 to 11.0; the 95th percentile at 20 floors is 28.0 vs 74.4 ticks. ETA is greedy: it minimises the arrival time for the caller in front of it, while Q-learning minimises the discounted waiting of everyone, and the long round trips in a tall building make those two goals differ more. Comparing choices on the 20-floor building (analyze_policy.py), the learned policy picks the same car as ETA on 64% of calls. Averaged over all calls, the car it sends is expected to arrive later (12.9 vs 9.5 ticks) and has a similar number of stops already planned (1.22 vs 1.31). So it accepts a slower pickup for some callers in exchange for shorter waits overall; we could not reduce what it learned to one simple rule. With only 6 floors trips are short and ETA is as good or better. Because our state describes cars relative to the call, the Q-table does not grow with height (971 to 2,781 states visited).

E3. Number of cars (up-peak, 10 floors)



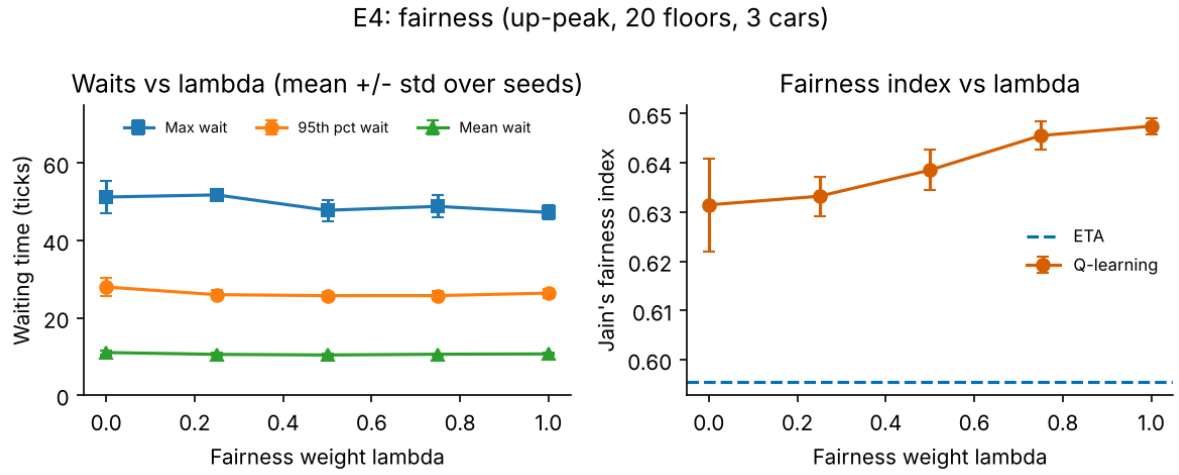
Cars	Nearest car	ETA	Q-learning	Gain vs ETA	Possible states	Visited	Coverage
2	58.8	6.8	7.0 +/- 0.1	-3%	1,872	527	100.0%
3	151.9	6.7	5.9 +/- 0.2	+12%	8,736	2,033	100.0%
4	203.7	7.0	6.4 +/- 0.2	+9%	32,760	5,938	99.9%
5	232.0	7.7	7.0 +/- 0.2	+10%	104,832	14,242	99.5%





With 2 cars there is little room for clever assignment and Q-learning roughly matches ETA (-3%). With more cars Q-learning is ahead (+12% with 3, +9% with 4, +10% with 5), but the gain does not keep growing. The cost shows up in learning speed: the number of states visited grows from 527 to 14,242, and the 5-car agent learns slowest (its learning curve starts far above ETA and only drops below it near the end of training). This is the curse of dimensionality: each extra car multiplies the situations the table must learn. Treating cars as interchangeable types keeps it manageable: labelling each car separately would allow about 5,971,968 states for 5 cars instead of 104,832.

E4. Fairness vs efficiency (up-peak, 20 floors, 3 cars)

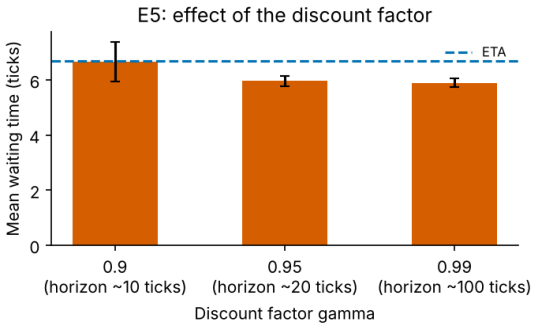


lambda	Mean wait	95th pct	Max wait	Jain index
0	11.0 +/- 0.5	28.0 +/- 2.3	51.2 +/- 4.3	0.631
0.25	10.6 +/- 0.3	26.0 +/- 1.1	51.8 +/- 0.7	0.633
0.5	10.4 +/- 0.4	25.7 +/- 1.0	47.8 +/- 2.7	0.639
0.75	10.6 +/- 0.2	25.7 +/- 1.3	48.8 +/- 2.9	0.646
1	10.7 +/- 0.2	26.4 +/- 1.2	47.2 +/- 1.8	0.647
ETA	31.0	74.4	99.8	0.595
Nearest car	149.0	286.7	312.0	0.718

Jain's index is computed on (wait + 1); it equals 1 when everyone waits equally long. Note that it measures equality, not quality: Nearest car scores well only because almost everyone waits a long time.

Raising lambda gives a small improvement in fairness at almost no cost in efficiency. Jain's index rises steadily (0.631, 0.633, 0.639, 0.646, 0.647 for lambda = 0, 0.25, 0.5, 0.75, 1), and the maximum wait falls from 51.2 +/- 4.3 to 47.2 +/- 1.8 ticks, while the mean wait stays between 10.4 and 11.0. The effect is modest and partly within seed-to-seed variation (the maximum wait is not monotone in lambda), and an earlier run during development showed no clear effect at all, so we treat it as weak evidence. The likely reason it is small is structural: the dispatcher only acts when a new call appears and can never reassign an old one, so it has little power to rescue someone who is already waiting. All Q-learning settings have a much shorter worst wait than ETA, whose maximum wait here is 99.8 ticks. Note that Jain's index measures how evenly waits are spread, not how short they are.

E5. Discount factor and horizon



A very short horizon hurts. Mean wait is 6.66 +/- 0.72, 5.96 +/- 0.19, 5.89 +/- 0.16 ticks for gamma = 0.9, 0.95, 0.99. With gamma = 0.9 (about 10 ticks of look-ahead) the agent barely beats ETA and varies a lot between seeds; gamma = 0.95 and 0.99 are both clearly better and within noise of each other. Our interpretation: a short horizon misses delayed effects such as overloading a busy car. Note that the gamma = 0.95 vs 0.99 ranking

changed between development runs, so we do not claim either is best.

3. Conclusions

• RL is worth it where the classical rule is short-sighted and decisions have large, delayed consequences: up-peak in tall buildings (64% lower mean wait than ETA at 20 floors). With a short building, few cars, or one elevator (E0), ETA or LOOK is already about as good.

• Tabular Q-learning with a building-wide reward fails when each decision has only a small effect (interfloor and down-peak traffic): a credit-assignment problem, not a shortage of data.

• Taller buildings clearly increase the benefit of learning. More cars help a little, but slow learning down as the state space grows; exploiting symmetry between cars keeps the table manageable.

• Weighting long waits more heavily in the reward gave only a small fairness gain, because assignment-only control cannot revisit old calls. Fairness needs actions that can prioritise long waiters, not just a different reward.

• A very short planning horizon (gamma = 0.9) clearly hurts; beyond about 20 ticks of look-ahead the differences were within noise.

4. Limitations

• Simplified simulator: fixed travel and door times, Poisson traffic, only the assigned car picks up a hall call, and calls are never reassigned.

• Coarse, hand-designed state (ETA and busyness buckets). States unseen in training fall back to ETA, although coverage was above 99% in all reported settings.

• One traffic intensity, three seeds per setting, and buildings of at most 20 floors and 5 cars because of compute. The smaller effects (the 10-floor up-peak gain, E4, E5) are close to the seed-to-seed variation and moved between development runs; the large effects (tall buildings, down-peak and interfloor losses) were stable.

• The agent is trained separately for each traffic profile and building, and is not tested on traffic that changes during the day.

5. Further work

• Function approximation (e.g. a small neural network or linear features) to replace the table and scale to more cars.

• Better credit assignment: per-car or per-passenger rewards instead of a single building-wide reward.

• Richer actions for fairness: reassigning waiting calls or explicit priority for long waiters, and constrained formulations (minimise mean wait subject to a maximum wait).

• Real traffic traces, time-varying traffic, and destination dispatch, where passengers enter their destination in the lobby.